

Binance Futures Order Bot - Report

1. Project Overview

This project is a CLI-based trading bot for Binance USDT-M Futures. It supports multiple order types, robust input validation, and structured logging. The bot is designed for automated trading and can be used on Binance's testnet or mainnet.

2. Features

Core Orders: Market and Limit orders.

Advanced Orders: Stop-Limit, OCO (One-Cancels-the-Other), TWAP (Time-Weighted Average Price), Grid orders.

Validation: Checks for valid symbols, quantities, and price thresholds.

Logging: All actions, errors, and executions are logged in bot.log with timestamps.

3. Setup Instructions

Clone the repository:

```
git clone https://github.com/Mythripaluri/binance_bot.git
cd binance_bot
```

Install dependencies:

```
pip install -r requirements.txt
```

Configure environment variables:

Create a .env file in the project root with the following content:

```
BINANCE_API_KEY=your_api_key_here
BINANCE_API_SECRET=your_api_secret_here
USE_TESTNET=true
DEFAULT_SYMBOL=BTCUSDT
POSITION_MODE=ONE_WAY
```

4. Usage Examples

Account Info

Check account details:

```
python -m src.main account
```

Check open orders for a symbol:

```
python -m src.main orders BTCUSDT
```

Check positions for a symbol:

```
python -m src.main positions BTCUSDT
```

Ping Binance and get server time:

```
python -m src.main ping
```

Get latest price for a symbol:

```
python -m src.main price BTCUSDT
```

Place Orders

Market Order:

```
python -m src.main order market --symbol BTCUSDT --side BUY --qty 0.01
```

Limit Order:

```
python -m src.main order limit --symbol BTCUSDT --side SELL --qty 0.01 --price 75000
```

Stop-Limit Order:

```
python -m src.main order stop-limit --symbol BTCUSDT --side BUY --qty 0.01 --stop 70000  
--price 70500
```

OCO Order:

```
python -m src.main order oco --symbol BTCUSDT --side BUY --qty 0.01 --tp 75000 --sl  
70000
```

Strategies

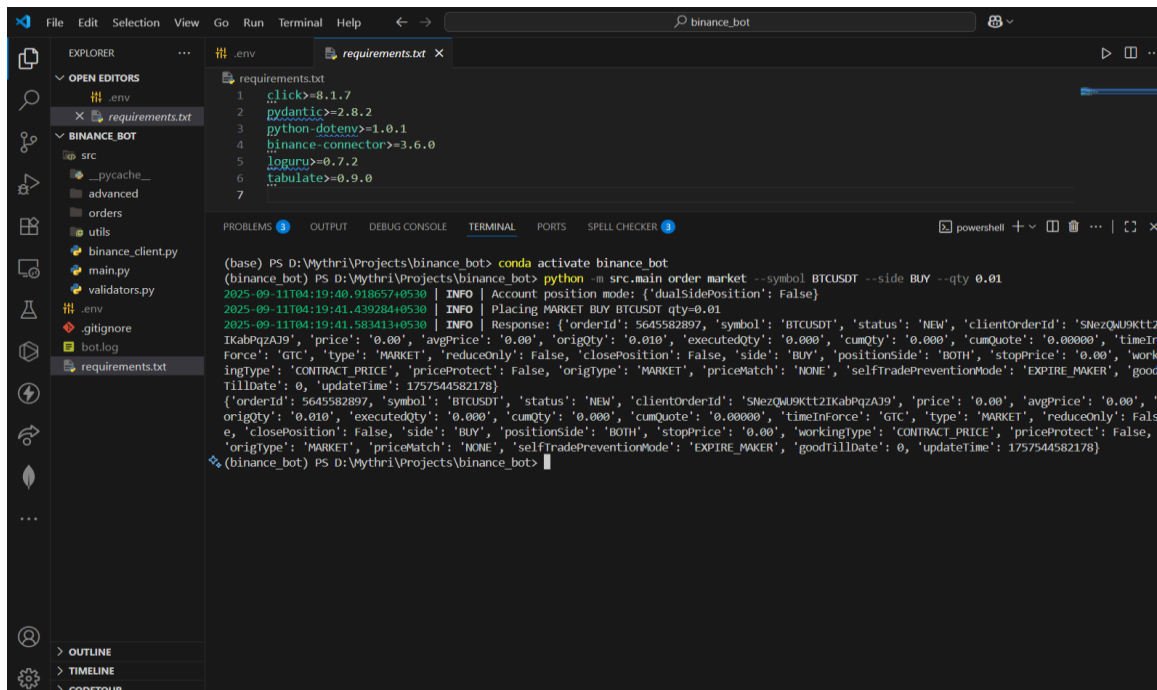
TWAP Order:

```
python -m src.main strat twap --symbol BTCUSDT --side BUY --qty 0.05 --slices 5 --  
duration 10
```

Grid Order:

```
python -m src.main strat grid --symbol BTCUSDT --qty 0.01 --lower 70000 --upper 75000 --levels 5
```

5. Screenshots

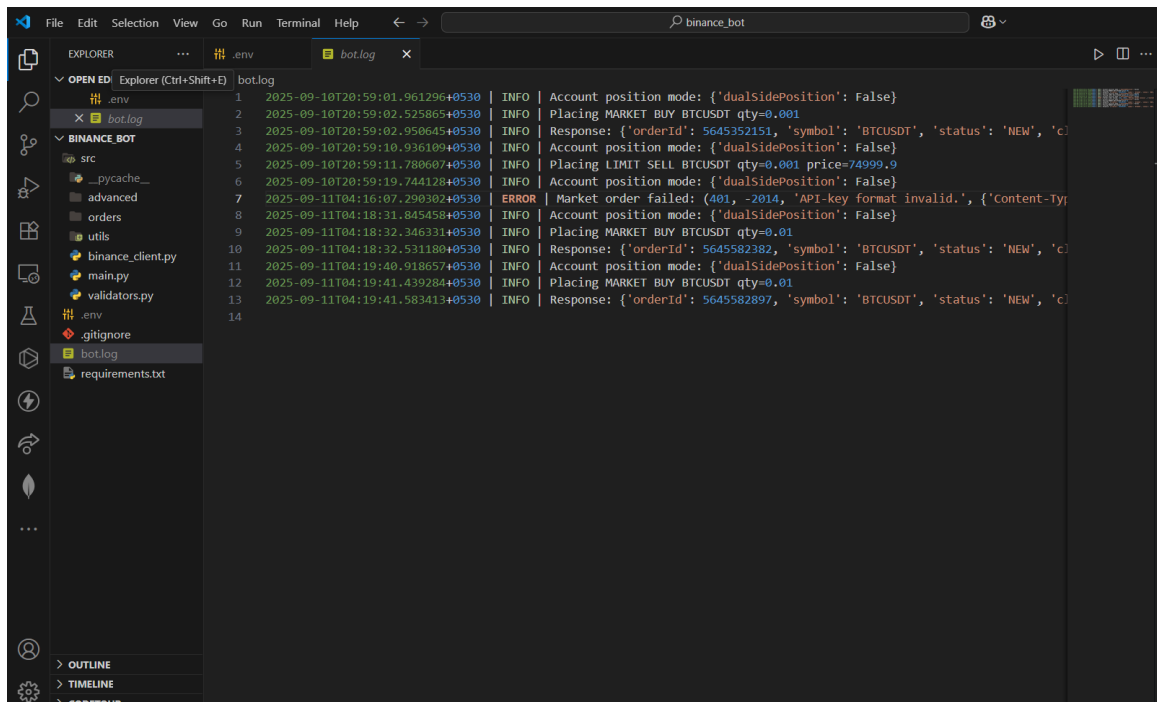


The screenshot shows the VS Code interface with the `requirements.txt` file open in the editor. The file contains the following dependencies:

```
click>=8.1.7
pydantic>=2.8.2
python-dotenv>=1.0.1
binance-connector>=3.6.0
loguru>=0.7.2
tabulate>=0.9.0
```

The terminal output shows the bot's initialization process:

```
(base) PS D:\Mythri\Projects\binance_bot> conda activate binance_bot
(binance_bot) PS D:\Mythri\Projects\binance_bot> python -m src.main order market --symbol BTCUSDT --side BUY --qty 0.01
2025-09-11T04:19:40.91865740530 | INFO | Account position mode: {'dualSidePosition': False}
2025-09-11T04:19:41.43928440530 | INFO | Placing MARKET BUY BTCUSDT qty=0.01
2025-09-11T04:19:41.58341340530 | INFO | Response: {'orderId': '5645582897', 'symbol': 'BTCUSDT', 'status': 'NEW', 'clientOrderId': 'SnezQAU9Ktt2IKabPqzAJ9', 'price': '0.00', 'avgPrice': '0.00', 'origQty': '0.010', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType': 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTillDate': 0, 'updateTime': 1757544582178}
(binance_bot) PS D:\Mythri\Projects\binance_bot>
```



The screenshot shows the VS Code interface with the `bot.log` file open in the editor. The log contains the following entries:

```
1 2025-09-10T20:59:01.96129640530 | INFO | Account position mode: {'dualSidePosition': False}
2 2025-09-10T20:59:02.52586540530 | INFO | Placing MARKET BUY BTCUSDT qty=0.001
3 2025-09-10T20:59:02.95064540530 | INFO | Response: {'orderId': '5645352151', 'symbol': 'BTCUSDT', 'status': 'NEW', 'clientOrderId': 'SnezQAU9Ktt2IKabPqzAJ9', 'price': '0.00', 'avgPrice': '0.00', 'origQty': '0.010', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType': 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTillDate': 0, 'updateTime': 1757544582178}
4 2025-09-10T20:59:10.93610940530 | INFO | Account position mode: {'dualSidePosition': False}
5 2025-09-10T20:59:11.78060740530 | INFO | Placing LIMIT SELL BTCUSDT qty=0.001 price=74999.9
6 2025-09-10T20:59:19.74412840530 | INFO | Account position mode: {'dualSidePosition': False}
7 2025-09-11T04:16:07.29030240530 | ERROR | Market order failed: (401, -2014, 'API-key format invalid.', {'Content-Type': 'application/json'})
8 2025-09-11T04:18:31.84545840530 | INFO | Account position mode: {'dualSidePosition': False}
9 2025-09-11T04:18:32.34633140530 | INFO | Placing MARKET BUY BTCUSDT qty=0.01
10 2025-09-11T04:18:32.53118040530 | INFO | Response: {'orderId': '5645582382', 'symbol': 'BTCUSDT', 'status': 'NEW', 'clientOrderId': 'SnezQAU9Ktt2IKabPqzAJ9', 'price': '0.00', 'avgPrice': '0.00', 'origQty': '0.010', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType': 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTillDate': 0, 'updateTime': 1757544582178}
11 2025-09-11T04:19:40.91865740530 | INFO | Account position mode: {'dualSidePosition': False}
12 2025-09-11T04:19:41.43928440530 | INFO | Placing MARKET BUY BTCUSDT qty=0.01
13 2025-09-11T04:19:41.58341340530 | INFO | Response: {'orderId': '5645582897', 'symbol': 'BTCUSDT', 'status': 'NEW', 'clientOrderId': 'SnezQAU9Ktt2IKabPqzAJ9', 'price': '0.00', 'avgPrice': '0.00', 'origQty': '0.010', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType': 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTillDate': 0, 'updateTime': 1757544582178}
14
```

6. Code Structure

The project is organized as follows:

src/

binance_client.py

main.py

validators.py

advanced/

grid.py

oco.py

twap.py

orders/

limit_orders.py

market_orders.py

shared.py

stop_limit.py

utils/

common.py

logger.py

7. Logging & Validation

All actions and errors are logged in bot.log with timestamps and details.

Input validation ensures only valid symbols, quantities, and prices are accepted.

Example log entry:

2025-09-10T20:59:02.525865+0530 | INFO | Placing MARKET BUY BTCUSDT qty=0.001

8. Testing

The bot was tested on Binance testnet using sample commands for each order type.

Logs and responses were verified for correctness.

9. Challenges & Learnings

Handling Binance API errors and edge cases.

Implementing advanced order types (OCO, TWAP, Grid).

Ensuring robust input validation and logging.

10. References

Binance Futures API Documentation: <https://binance-docs.github.io/apidocs/futures/en/>

Python Binance Library: <https://python-binance.readthedocs.io/en/latest/>